

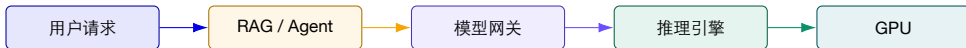
面向 AI 原生系统的智能运维

第 5 讲 | 智能监控与可观测性工程 (3/3)

GPU、Token、推理延迟、模型质量、成本与 Agent 调用链

陈鹏飞 | 中山大学

从第 4 讲继续：把“调用路径”延伸到 Token 与 GPU



第 3 讲 Metrics 与 OpenTelemetry 建立数据采集与存储。

第 4 讲 Logs 与 Traces 还原跨服务证据链。

第 5 讲 把 Token、质量、成本和 GPU 状态挂到同一条链。

Goodput 算例：吞吐更高的配置为什么可能更差？

比较两个配置的同一 60 秒窗口；请求合格需同时满足延迟门槛与任务质量条件。

配置	完成请求	同时合格请求	平均功率
A	600	480	0.6 kW
B	720	432	0.8 kW
计算 Goodput	A: 480/60	B: 432/60	分别为 8 与 7.2 请求/s

$$E_A = \frac{600/60}{480} = 0.0208 Wh/合格请求, \quad E_B = 0.0309 Wh/合格请求$$

推导与判断 B 的完成吞吐提高 20%，但 Goodput 下降 10%，单位合格请求能耗更高。这里比较的是请求级 Goodput，不能直接替代不同输出长度下的 Token 吞吐。

算例使用假设数据。

开场事故：GPU 很忙、请求没报错，用户为什么仍然不满意？

监控面板

- ▶ GPU Util 长期 92%，显存使用 88%；
- ▶ HTTP 成功率 99.8%，吞吐提升 24%；
- ▶ 每分钟 Token 数持续增长；
- ▶ 没有明显 Pod 重启或 5xx 峰值。

用户与财务反馈

- ▶ 首 Token 等待从 0.8s 上升到 6.2s；
- ▶ Agent 偶发执行 20 多次工具调用；
- ▶ RAG 回答速度更快但引用错误；
- ▶ 单次成功任务成本上涨 3.7 倍。

关键矛盾 资源忙、请求成功、Token 增长，都不等于用户任务在 SLO 内高质量完成。

先不要看 Dashboard：把问题写成四个可验证问题

Q1 | 慢在哪里？ 排队、Prefill、首Token、Decode、工具调用还是检索？

Q2 | 资源为何忙？ 有效计算、显存搬运、KV Cache、调度开销还是硬件降频？

Q3 | 贵在哪里？ 长 Prompt、长输出、重试、循环、昂贵模型或低利用率？

Q4 | 质量为何下降？ 检索、Prompt、模型、工具结果、环境状态还是评测失真？

症状可量化

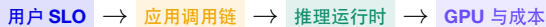
路径可追踪

成本可归因

质量可验证

本讲路线图

1. AI 系统指标模型：调用链、TTFT/TPOT、分位数、吞吐与 Goodput；
2. GPU 与推理运行时：DCGM、显存、KV Cache、调度、Prefill/Decode；
3. Token、成本与 Langfuse：vLLM 指标、OTel 语义、跨层 Trace；
4. 模型质量、RAG 与 Agent：评测、反馈、工具调用和 Outcome Validity；
5. 实验 1：部署多层监控、注入压力并完成依据实际数据进行诊断。



本讲诊断要求：每个结论必须回答“哪条请求、哪个阶段、哪个资源”

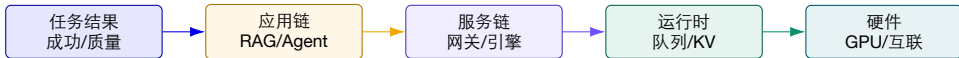
问题	至少需要的观测	可接受结论示例
TTFT 变慢 TPOT 变慢 成本上涨 质量下降 GPU 异常	request/trace ID、queue、prefill、prompt tokens decode time、running batch、KV、GPU/DRAM model、input/output tokens、重试/工具次数 版本、检索结果、输出、Score、业务反馈 pod/GPU UUID、XID/ECC、温度、时钟、链路	长 Prompt 租户导致 Prefill 队列拥塞 Decode 受显存带宽与并发批次干扰 Agent 循环让成功任务平均调用模型 8.4 次 新索引降低某类查询的 context recall GPU 降频且出现 XID，需隔离设备并复测

反例 “GPU 利用率高导致延迟高” 缺少请求范围、阶段、因果证据和验证动作，不能算完整诊断。

一、从用户体验建立 AI 系统指标模型

先定义“好请求”，再决定采集哪些基础设施和模型指标。

AI 系统可观测性至少有五个平面



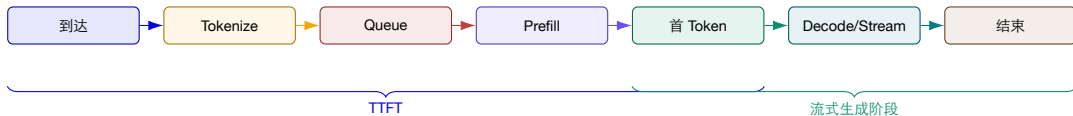
外层决定价值 任务是否完成、是否正确、是否值得这个成本。

中层解释请求 调用次数、阶段延迟、模型/工具/检索路径。

内层解释资源 队列、批处理、KV、算力、带宽和硬件健康。

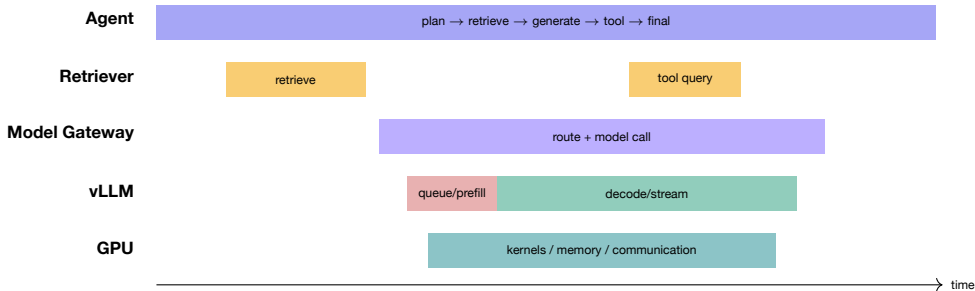
可观测性不是五套孤立 Dashboard，而是同一请求在五个平面的不同投影。

先画请求生命周期：延迟不是一个不可分解的数字



测量边界 客户端 TTFT 包含网络与网关；引擎 TTFT 只解释服务端。两者不能混用，必须明确时钟与边界。

把应用调用链与推理阶段放进同一个 Trace



统一身份 Trace ID 连接应用步骤；Span 属性记录模型、Token、租户和版本；Resource 属性连接 Pod、GPU UUID 与部署。

RED 与 USE 仍然有用，但需要 AI 语义扩展

方法	传统问题	AI 扩展	示例
RED	Rate	请求与 Token 速率	requests/s、input/output tokens/s
RED	Errors	失败与不完整结束	5xx、abort、length、tool error、无有效答案
RED	Duration	多阶段延迟	TTFT、TPOT、queue、prefill、decode、tool latency
USE	Utilization	计算/显存/队列利用	GPU/SM/DRAM、KV blocks、running slots
USE	Saturation	等待与背压	waiting requests、queue time、pre-emption
USE	Errors	设备与链路健康	XID、ECC、row remap、PCIe replay、NVLink

扩展项 RED/USE 不能直接表达 “回答正确吗、任务完成吗、每个成功任务花多少钱”，还需 Quality、Outcome 与 Cost。

同一个症状可能来自完全不同的层

症状	应用层解释	推理运行时解释	GPU/硬件解释
TTFT 上升 TPOT 上升 成本上升 质量下降	RAG 检索慢、路由重试 多 Agent 并发抢占 循环、重试、工具爆炸 Prompt/索引/工具数据变化	waiting 增长、长 Prompt Prefill batch 变化、Decode 干扰 输出变长、缓存命中下降 截断、length finish、超时	计算降频、PCIe 传输异常 DRAM 活跃、功耗/热限制 低利用率造成 GPU-hour 浪费 通常不是首要解释，除非硬件错误
GPU Util 下降	流量下降、外部 API 慢	调度空洞、CPU 前后处理	设备异常或链路阻塞

选择一行，写出至少两条能够区分三种解释的观测证据。

指标不是事实全集，而是被压缩过的诊断假设

一个 Gauge 看不到什么

- ▶ 请求之间的分布与长尾；
- ▶ 指标变化对应哪些 Trace；
- ▶ 哪种 Prompt、租户或模型受影响；
- ▶ 高利用率来自有效工作还是浪费；
- ▶ 质量与业务结果是否改善。

因此要保留

- ▶ Histogram：延迟与 Token 分布；
- ▶ Counter：累计工作与错误事件；
- ▶ Trace：单请求因果与调用结构；
- ▶ Logs/Events：决策与异常细节；
- ▶ Eval/Feedback：延迟到达的质量标签。

原则 指标用于发现和比较；Trace 用于归因；评测与反馈用于判断系统是否真的“更好”。

用户感知至少包含四种时间

响应开始 TTFT: 多久看到第一个 Token。

阅读流畅 平均 TPOT 描述生成速度;
ITL 分布描述相邻 Token 间隔是否稳定。

任务结束 E2E: 拿到完整响应或任务最终结果。

交互完成 Agent 可能多轮思考、工具调用和确认, 远长于单次模型 E2E。

Chat: TTFT 很敏感

代码生成: TPOT 与总时长

Agent: 任务完成时长与步骤数

TTFT: 首 Token 之前的等待都可能被用户感知

$$TTFT_{client} = t_{\text{first token received}} - t_{\text{request sent}}$$

常见主导因素

- ▶ Prompt 长度与 Prefill (含首 Token 采样);
- ▶ waiting 队列与并发调度;
- ▶ 冷启动、模型加载与前缀缓存;
- ▶ RAG/Agent 在模型调用前的步骤。

测量陷阱

- ▶ 非流式客户端无法直接测到首 Token;
- ▶ 同一侧用单调时钟; 另查网络、网关与队列;
- ▶ 只看平均值掩盖长 Prompt 与高并发;
- ▶ SDK 缓冲可能改变首块数据时间。

TPOT 与 ITL：流式体验不只是一个平均速度

$$\text{Average TPOT} = \frac{T_{last\ token} - T_{first\ token}}{N_{output} - 1} \quad \text{ITL}_i = T_i - T_{i-1}$$

平均 TPOT 适合容量与模型/硬件比较，但会掩盖抖动。

ITL 分布 能发现周期性停顿、批调度抖动和网络块传输。

P95/P99 TPOT 表示请求平均生成耗时的长尾；单次停顿仍要看 ITL 分布。

分段分析 按模型、输出长度、并发、租户和流式协议切片。

仅对输出至少 2 个 Token 的请求定义 TPOT；网络输出块不一定等于单个 Token，须注明测量口径。

单次生成与完整任务：先确定计时边界

$$T_{\text{request to last token}} = TTFT + (N_{\text{output}} - 1) \times TPOT$$

公式成立需要

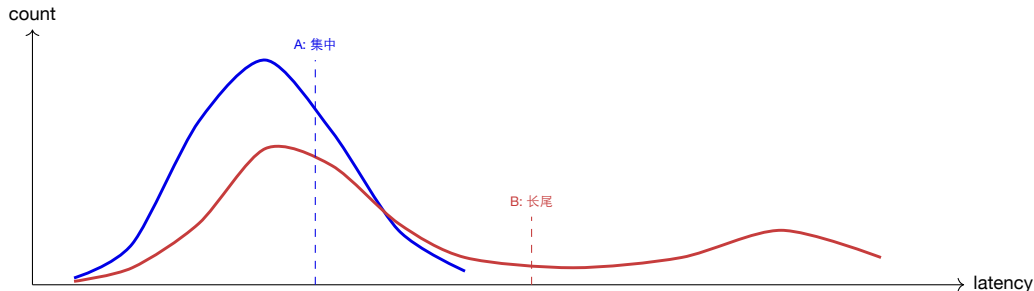
- ▶ 仅考虑单次模型调用；
- ▶ TPOT 使用同一请求的平均值；
- ▶ 终点是最后 Token 到达，不含后处理；
- ▶ 至少输出 2 个 Token，计数与时钟一致。

完整任务另行计时

- ▶ 检索若已计入应用 TTFT，不再重复相加；
- ▶ 多次模型调用；
- ▶ 工具执行与外部 API；
- ▶ 状态持久化、验证与人工确认。

关键判断 不要用单次推理延迟代替“任务完成时间”；Agent 的用户 SLO 必须定义在任务边界。

为什么必须看分布：平均 1 秒可能对应两种完全不同的系统



系统 A 大多数请求接近均值，容量规划和 SLO 都较稳定。

系统 B 多数很快、少数极慢；平均值可能相同，用户体验完全不同。

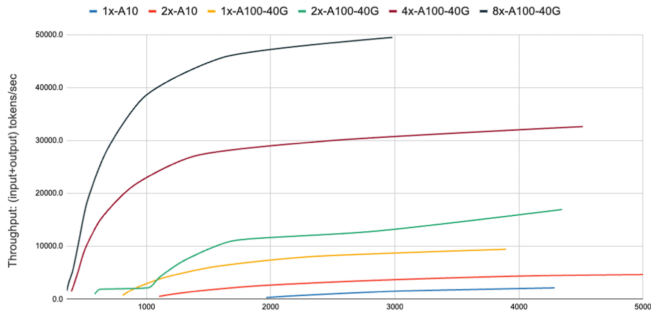
Histogram bucket 必须覆盖目标 SLO；否则 P99 会被最外层 bucket 截断，得出错误结论。

分位数不切片，仍然可能把根因平均掉

维度	为什么要切片	基数治理
model / version	模型架构、量化和版本不同	使用受控枚举，不放完整路径哈希
prompt bucket	Prefill 与 KV 压力依赖长度	用长度区间，不用原始 Token 数作标签
tenant / tier	流量、预算与 SLO 不同	大租户白名单，其余聚合为 other
finish reason	stop、length、abort 含义不同	小型枚举，适合 Counter 标签
route / experiment	路由策略与 A/B 变化	使用稳定实验 ID，不放请求 ID
GPU / pod	设备或实例局部异常	指标标签保留 UUID；请求 ID 放 Exemplar/Trace

规则 用于聚合的低基数字段放 Metric 标签；高基数请求身份放 Trace 与 Exemplar。

吞吐—延迟不是单点：系统沿一条饱和曲线运行

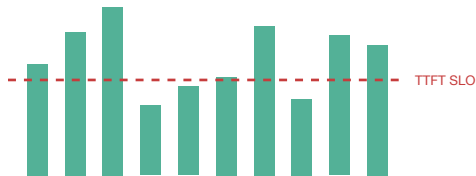


读图方法

- ▶ 增加并发通常先提高吞吐；
- ▶ 接近饱和点后，延迟快速上升；
- ▶ 不同 GPU 数量形成不同曲线；
- ▶ 容量选择必须带延迟约束。

错误目标 只最大化 tokens/s，可能把系统推到用户无法接受的区域。

高吞吐不等于高 Goodput



10 个请求完成
只有 4 个满足 SLO

$$\text{Throughput} = \frac{\#completed}{time}$$

$$\text{Goodput} = \frac{\#completed\ within\ SLO}{time}$$

- ▶ SLO 可以同时约束 TTFT 与 TPOT;
- ▶ 还可加入质量、成本或成功任务条件;
- ▶ Goodput 直接惩罚“完成但不可用”的请求。

为不同请求定义 SLO 矩阵，而不是一个全局延迟阈值

请求类型	TTFT	TPOT	质量/结果	成本约束
交互问答	P95 < 1s	P95 < 50ms	引用正确、无严重幻觉	每次对话预算
代码生成	P95 < 2s	P95 < 35ms	测试通过/编译成功	每个通过任务成本
批量摘要	可放宽	可放宽	覆盖率与事实一致	tokens/GPU-hour
RAG 检索	P95 < 1.5s	P95 < 60ms	recall/groundedness	每个有效答案成本
运维 Agent	P95 < 3s	不必极低	工具成功、状态验证	每次任务成本/步数

注意 Goodput 的分母是时间，分子应是满足该请求类别全部关键条件的完成请求。

SLO 设计的四个常见错误

只看 **E2E** 无法区分首 Token 慢与流式卡顿。

只看 **P50** 长尾用户持续受损，面板仍然“绿色”。

跨场景共用阈值 交互、批处理和 Agent 的体验目标不同。

忽略质量/成本 快速返回错误答案，或高成本完成任务，也不应算 Goodput。

边界明确

分布明确

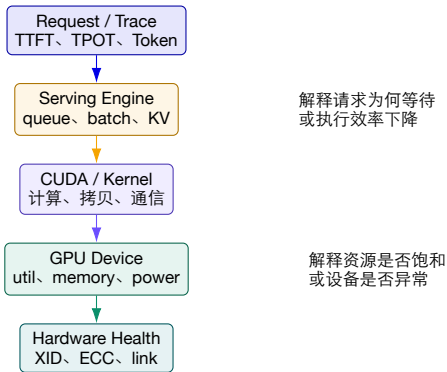
场景明确

结果明确

二、从推理运行时下钻到 GPU

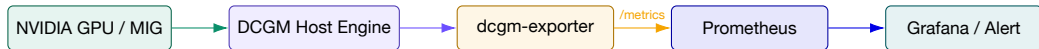
GPU 指标只有与请求阶段、队列和 KV Cache 对齐后才具有诊断意义。

GPU 可观测性是一个分层测量栈



诊断顺序 从请求症状向下钻；不要从某个 GPU 指标异常直接反推所有请求。

DCGM Exporter: 每个 GPU 节点暴露 Prometheus 指标



Kubernetes 常用 DaemonSet
并映射 Pod/容器标签

生产注意 一个节点通常运行一个 exporter; MIG、Pod 映射、字段支持和采样周期需与 GPU 型号/驱动/DCGM 版本核对。

GPU 指标按“忙、满、慢、坏”四类组织

问题	代表指标	能说明什么	不能单独说明什么
忙	GPU_UTIL、GR_ENGINE_ACTIVE、 TENSOR_ACTIVE	设备执行活跃程度	工作是否有效、用户是否满足 SLO
满	FB_USED/FREE、KV 使用、run- ning/waiting	显存或服务槽位压力	内存具体被权重、KV 还是碎片占 用
慢	SM/MEM_CLOCK、POWER、TEMP、 DRAM/PCIe	降频、带宽或传输限制线索	哪条请求、哪个阶段受影响
坏	XID、ECC、row remap、PCIe replay、 NVLink error	设备/链路可靠性风险	业务影响范围和恢复是否成功

DCGM 默认配置并不会启用所有高开销 profiling 字段；按硬件支持、诊断价值和采集成本选择。

为什么 GPU Util=95% 仍然可能是坏消息？

可能是健康高利用率

- ▶ Goodput 高且 TTFT/TPOT 在 SLO 内；
- ▶ batch 稳定，waiting 可控；
- ▶ Token/s 与 GPU-hour 经济性良好；
- ▶ 无频率下降、错误或重试。

也可能是低效高利用率

- ▶ 大量请求已超 SLO，仍在继续执行；
- ▶ Agent 循环生成无效 Token；
- ▶ 长 Prompt 挤占 Prefill，短请求排队；
- ▶ 频繁重算、调度开销或无效批次。

结论 GPU Util 是资源状态，不是业务价值；必须同时看 Goodput、Token 有效率和请求 Trace。

GPU Util 很低也不等于“流量不足”

模式	伴随证据	可能根因
GPU 低、 waiting 高	CPU 高、tokenize/metadata 慢	CPU 前后处理或调度瓶颈
GPU 低、外部 Span 长	Retriever/Tool/API 占关键路径	GPU 在等待应用层依赖
GPU 低、 PCIe RX/TX 异常	Host-device 拷贝多、链路重试	数据传输或拓扑问题
GPU 周期性低谷	batch 周期、同步点明显	调度空洞或阶段屏障
GPU 低、时钟/温度异常	clock 下降、thermal violation	热/功耗限制或设备健康
GPU 低、无请求	arrival rate 同时下降	真实流量不足或过度配置

诊断方法 先比较 arrival、waiting、running，再关联 CPU/网络/外部 Span，最后判断是否需要扩容。

显存“满”之前，先问显存被谁占用



静态与半静态

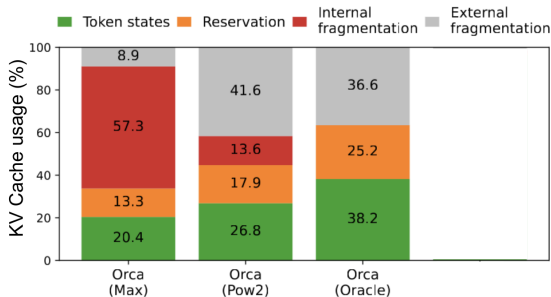
- ▶ 权重：模型大小、精度、并行方式；
- ▶ Runtime/Workspace：Kernel 与框架；
- ▶ CUDA context、通信 buffer。

随工作负载变化

- ▶ KV Cache：上下文长度与并发请求；
- ▶ 临时激活与采样状态；
- ▶ 分配策略导致的内部/外部碎片。

因此 FB_USED 只能告诉你总量，不能代替引擎级 KV block、请求长度和分配效率指标。

研究证据：传统 KV Cache 分配可能只让 20–40% 空间存放有效 Token 状态



图中浪费来源

- ▶ 预留但尚未使用；
- ▶ 内部碎片；
- ▶ 外部碎片；
- ▶ 请求长度不可预测。

结论 显存使用率高，可能包含大量不可用于新请求的碎片与预留。

KV Cache 监控要同时看容量、效率与请求压力

观测	解释	联合判断
kv_cache_usage_perc prompt length histogram generation length histogram	已使用 block 比例 输入上下文分布 输出长度分布	高使用 + waiting 上升：容量压力 长 Prompt 增多：Prefill 与 KV 同时受压 长输出占用 block 更久、提高在途请求
running / waiting preemption / recompute GPU FB used/free	执行与排队请求数 请求被抢占或重算 设备总显存	running 稳定、waiting 上升：已饱和 KV 不足或调度策略导致额外工作 用于验证引擎视角与设备视角是否一致

警报不是阈值拼接 “KV > 90%” 应结合 waiting、TTFT、长 Prompt 占比和 Goodput，避免把健康高负载误报成故障。

Prefix Cache: 命中率必须与节省的 Prefill 工作一起解释

$$\text{Hit Rate} = \frac{\Delta \text{prefix_cache_hits}}{\Delta \text{prefix_cache_queries}}$$

命中率升高可能带来

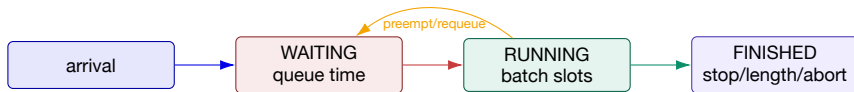
- ▶ 重复系统 Prompt 避免重复 Prefill;
- ▶ TTFT 与计算量下降;
- ▶ 同样 GPU 可承载更多请求。

但不能只看命中率

- ▶ 短前缀高命中，节省可能很小;
- ▶ 缓存占用也会竞争 KV 容量;
- ▶ Prompt 版本变化可能造成命中骤降;
- ▶ 不同租户不可错误共享敏感上下文。

更好的面板 Hit rate + cached tokens + TTFT 改善 + KV 占用 + Prompt 版本。

队列状态把“流量增长”转换成“用户等待”



arrival > **service**
waiting 与 queue time 持续上升。

batch 已满 running 稳定
在上限，TTFT 主要受排队影响。

频繁抢占 额外重算/换入
换出，TPOT 与成本恶化。

Prefill 与 Decode 的资源特征不同

Prefill

- ▶ 一次处理完整输入序列；
- ▶ 大矩阵计算并行度高；
- ▶ Prompt 越长，计算量与 TTFT 越高；
- ▶ 通常更偏计算密集。

Decode

- ▶ 每步生成一个或少量 Token；
- ▶ 反复读取权重与 KV Cache；
- ▶ 批量大小影响算术强度；
- ▶ 常更受显存带宽与调度影响。

可观测性要求 至少分别采集 request prefill time、decode time、TTFT 与 inter-token latency，不能只保留一个 E2E。

Prefill/Decode 混部会产生请求间干扰

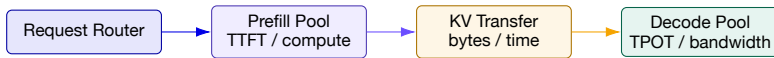


对新请求 长 Prefill 占据计算，影响其他请求的 TTFT 与调度。

对在途请求 Decode 被阻塞或批次变化，ITL/TPOT 出现尖峰。

因此需要把每个时间窗的 Prefill/Decode 负载比例与 TTFT/TPOT 同时展示。

P/D 解耦后，监控也必须按阶段拆分



Prefill Pool queue、
prompt tokens、
compute、TTFT。

KV Transfer 传输字节、
带宽、等待、失败与重试。

Decode Pool running、
output tokens、TPOT、
DRAM 活跃。

新故障面 解耦降低干扰，但增加路由、KV 传输和跨池容量匹配问题。

硬件可靠性指标：从“性能慢” 区分 “设备坏”

指标/事件	类型	处置语义
DCGM_FI_DEV_XID_ERRORS	最近 XID 值	查询 NVIDIA XID 含义，关联 Pod/作业并决定隔离/重启
ECC SBE/DBE totals	Counter	单比特可纠正但需趋势；双比特通常更严重
retired pages / row remap	Counter/Gauge	内存单元退化，关注 pending 与 remap failure
thermal/power violation	Counter	设备因温度/功耗约束降频的累计时长
SM/MEM clock	Gauge	与基线比较，确认是否发生降频
GPU health / XID totals	Exporter 指标	适合窗口内告警，但需核对 exporter 版本支持

证据链 硬件事件时间 → GPU UUID → 节点/Pod → 受影响 Trace → 隔离后 SLO 恢复。

互联指标：多 GPU 系统的瓶颈可能不在 GPU 核心

主机—设备

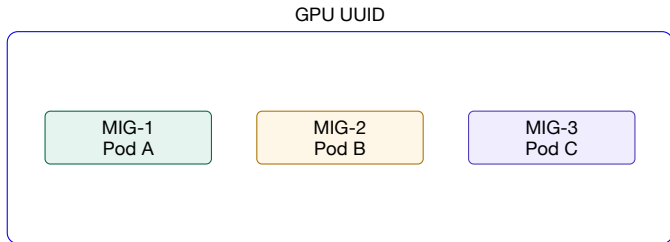
- ▶ PCIe RX/TX bytes/s;
- ▶ PCIe replay counter;
- ▶ Host-device copy 与 CPU pinned memory;
- ▶ 模型加载、KV 迁移、输入搬运。

设备—设备

- ▶ NVLink bandwidth;
- ▶ CRC / replay / recovery errors;
- ▶ Tensor/Pipeline Parallel 通信;
- ▶ P/D 解耦的 KV 传输路径。

诊断提示 GPU Util 下降 + 通信 Span 拉长 + PCIe/NVLink 异常，比单独的低 GPU Util 更接近可操作根因。

MIG 与 Kubernetes：资源归属必须精确到实例



- ▶ Node-level GPU 指标可能把多个 MIG 实例/Pod 平均在一起；
- ▶ 标签至少包含 cluster、node、GPU UUID、MIG UUID、namespace、pod、container；
- ▶ Trace Resource 属性应能映射到服务实例与 GPU 资源；
- ▶ Pod 重建会改变名称/UID，归因系统要保留时间有效性。

常见误判 某租户的请求慢，却用整卡平均指标解释；共享 GPU 场景下这通常不够。

GPU 标签丰富，但每个标签都会创建新的时序

适合 Metric 标签

- ▶ cluster / node;
- ▶ gpu_uuid / mig_uuid;
- ▶ namespace / pod / container;
- ▶ model / deployment / environment;
- ▶ 小型错误类型枚举。

不适合 Metric 标签

- ▶ request_id / trace_id;
- ▶ 完整 Prompt、用户原文;
- ▶ 任意 Tool 参数;
- ▶ 动态 URL、文件名和异常堆栈;
- ▶ 每个 Token 的序号。

实现方法 高基数字段进入 Trace/Logs; Metric 通过 Exemplar 跳到代表性请求。

TELLER: GPU 错误如何映射到请求

问题

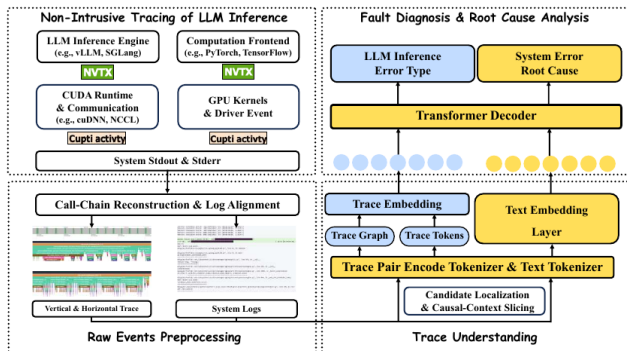
- ▶ LLM 推理由 Engine、CUDA Runtime、Driver 与 Kernel 共同执行；
- ▶ 系统日志与 GPU Trace 分散；
- ▶ 并发请求共享 Kernel，单按时间对齐容易误归因；
- ▶ 侵入式插桩会改变系统行为与开销。

方法

- ▶ NVTX + CUPTI 非侵入式采集；
- ▶ 重建垂直/水平调用链；
- ▶ Trace 与日志对齐；
- ▶ Transformer 解码器识别错误类型和根因。

本讲视角 TELLER 说明设备级证据必须投影回请求级 call-chain，才适合诊断与分析。

TELLER: 从 NVTX/CUPTI 原始事件重建跨层证据



读图顺序

1. 非侵入式采集；
2. 调用链重建；
3. 与系统日志对齐；
4. Trace 图/Token 编码；
5. 错误类型与根因输出。

诊断问题 如果只有 GPU_UTIL，图中哪些诊断信息会完全丢失？

TELLER 案例：不要把一次 Kernel 异常泛化成“整卡过载”

粗粒度观测

- ▶ GPU Util 89%;
- ▶ 请求错误率短时上升;
- ▶ 日志出现 CUDA error;
- ▶ 多个模型共享同一 GPU。

跨层证据

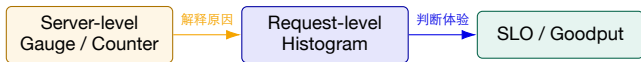
- ▶ 仅某类请求的特定 Kernel 链异常;
- ▶ CUPTI correlation 绑定 Host 调用与设备事件;
- ▶ 日志对齐到对应 request call-chain;
- ▶ 其他模型请求保持正常，排除整卡饱和。

验证动作 隔离故障请求/模型版本并复测；同时观察错误 Trace、Kernel 事件与请求 SLO 是否恢复。

三、Token、成本与 Lang-fuse 调用链

把每一次模型、检索和工具调用变成可聚合、可归因、可追踪的经济单元。

如何阅读 vLLM 监控指标：服务状态解释请求 SLO



Server-level running/waiting、KV usage、cache hits、Token counters。

Request-level TTFT、ITL/TPOT、E2E、queue、prefill、decode、长度分布。

vLLM 官方设计文档明确区分两类指标：引擎指标解释请求指标，请求指标通常直接对应 SLO。

vLLM stable 关键指标映射

Metric	类型	诊断用途
vllm:num_requests_running / waiting	Gauge	执行槽位与排队压力
vllm:kv_cache_usage_perc	Gauge	KV block 容量压力
vllm:prompt_tokens_total	Counter	Prefill 工作量与输入用量
vllm:generation_tokens_total	Counter	输出吞吐与用量
vllm:time_to_first_token_seconds	Histogram	TTFT 分布
vllm:inter_token_latency_seconds	Histogram	ITL/TPOT 分布
vllm:e2e_request_latency_seconds	Histogram	单次模型请求总延迟
vllm:request_queue_time_seconds	Histogram	排队时间
vllm:request_prefill_time_seconds	Histogram	Prefill 时间
vllm:request_decode_time_seconds	Histogram	Decode 时间
vllm:request_success_total	Counter	stop/length/abort 结束原因

指标名称按 2026-08 的 stable 文档校正；升级 vLLM 时应根据运行版本的 /metrics 实际输出核对。

PromQL: 计算 TTFT P95, 并保留模型维度

```
histogram_quantile(  
  0.95,  
  sum by (le, model_name) (  
    rate(vllm:time_to_first_token_seconds_bucket[5m])  
  )  
)
```

必须保留 **le** Histogram quantile 需要 bucket 上界标签。

窗口选择 5m 适合快速发现；低流量服务要更长窗口或记录规则。

进一步按 deployment、route 或 prompt bucket 切片；避免把高基数 tenant 全部直接放入聚合。

PromQL：把排队、到达率和完成率放在一起

等待请求

```
sum by (model_name) (  
    vllm:num_requests_waiting  
)
```

输出 Token 速率

```
sum by (model_name) (  
    rate(vllm:generation_tokens_total[5m])  
)
```

完成请求速率

```
sum by (model_name) (  
    rate(vllm:request_success_total[5m])  
)
```

联合解释 waiting 上升、Token/s 已饱和、完成率不再增长：服务接近容量边界。

Token 分布是性能、成本和滥用的共同解释变量

Input Tokens

- ▶ 决定 Prefill 工作量与 TTFT;
- ▶ 占用 KV Cache;
- ▶ 受历史对话、RAG chunks、系统 Prompt 影响;
- ▶ 极长输入可能来自滥用或拼接错误。

Output Tokens

- ▶ 决定 Decode 时间与流式成本;
- ▶ length finish 暗示截断或循环;
- ▶ Agent 中多次模型调用会累积;
- ▶ 不同任务的合理长度差异很大。

面板建议 Input/Output Token heatmap + P50/P95/P99 + 模型/路由/任务类型切片。

“成功请求”也要拆分结束原因

stop 模型正常遇到停止条件；仍需质量验证。

length 达到最大长度，可能截断、循环或预算不足。

abort 客户端取消、超时、调度或系统终止。

- ▶ **length** 比例突增：检查 `max_tokens`、Prompt 变化和 Agent 循环；
- ▶ **abort** 与 TTFT 高度相关：用户可能在首 Token 前放弃；
- ▶ HTTP 200 + **length** 不应自动计入高质量 Goodput；
- ▶ 结束原因应进入 Trace、Counter 与成本归因。

vLLM stable 的 request success counter 可按 finish reason 区分 stop、length、abort。

流式健康：Token/s 高，仍要检查首块与块间抖动

观测	健康模式	异常模式
TTFT	负载增加时缓慢上升	waiting 激增后陡升
ITL/TPOT	分布集中、少量长尾	周期性停顿、P99 抖动
output tokens/s	与并发平滑增长	高但 Goodput 下降
client disconnect	稳定低位	TTFT/ITL 恶化时上升
chunk size / buffering	协议与 SDK 一致	代理缓冲造成“假 TTFT”或突发块

端到端验证 服务端 Token 产生时间与客户端收到时间都要测；中间代理可能改变流式体验。

成本有两种模型：供应商账单与自托管资源成本

API / 商业模型

- ▶ 按 input/output/cached/reasoning 等单位计价；
- ▶ 价格随模型、区域、服务等级变化；
- ▶ 以供应商 usage 与账单口径优先；
- ▶ 价格表必须版本化和可审计。

自托管模型

- ▶ GPU-hour、能耗、CPU、内存、存储和网络；
- ▶ 空闲与过度配置也是成本；
- ▶ 共享 GPU 需要租户/请求分摊；
- ▶ Goodput 比单纯 Token 成本更有价值。

本课不写死价格 商业价格高度时变；实验使用可替换的版本化价格表或自托管估算参数。

供应商调用成本：优先使用响应中的精确 usage

$$C_{request} = N_{input} \cdot P_{input} + N_{output} \cdot P_{output} + \sum_k N_k \cdot P_k$$

记录字段

- ▶ provider、request/response model;
- ▶ input/output/cached/reasoning usage;
- ▶ price table version / contract ID;
- ▶ currency、region、service tier。

常见偏差

- ▶ 本地 tokenizer 与账单 Token 不一致;
- ▶ 推理模型隐藏 reasoning tokens;
- ▶ 重试/回退被遗漏或重复计费;
- ▶ 价格更新只影响新事件，历史需保留版本。

Langfuse 官方文档：已摄取 usage/cost 优先于推断值；自定义模型和私有价格应显式配置。

自托管成本：从 GPU 时间分摊到“每个成功任务”

$$C_{window} = C_{GPU-hour} + C_{CPU} + C_{memory} + C_{storage} + C_{network} + C_{ops}$$

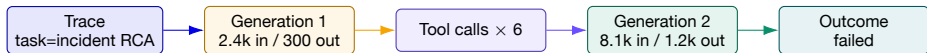
$$C_{request} \approx \frac{GPU-seconds_{request}}{GPU-seconds_{window}} \times C_{window}$$

近似分摊 可用 Token、占用时间、batch 份额或 trace-level GPU time 估计。

价值指标 **Cost per Good Request / Successful Task** 比 Cost per Token 更接近业务。

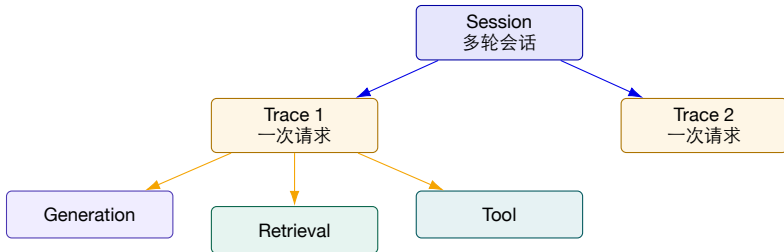
共享批次中精确归因很难，应标明分摊规则与不确定性；不要把估算值伪装成账单事实。

把成本挂到 **Trace**：先定位“哪一步烧钱”



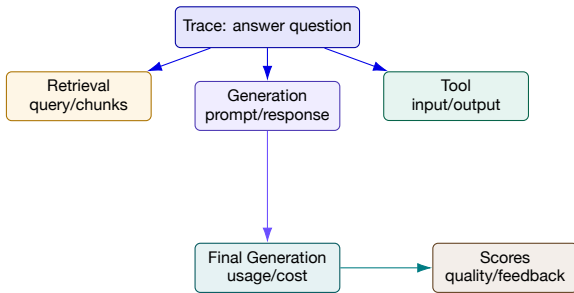
- ▶ Span/Observation 记录 model、usage details、cost details、latency 与结果；
- ▶ Trace 聚合总 Token、总成本、模型调用数、工具调用数和成功状态；
- ▶ 失败任务的成本不能从统计中删除，否则会低估真实单位经济性；
- ▶ 路由优化应比较“质量约束下的成功任务成本”。

Langfuse 数据模型：Observation、Trace 与 Session



核心语义 Observation 是 LLM、Tool、Retrieval 等步骤；Trace 是一次完整请求；Session 聚合多轮交互。

Langfuse 调用链：应用层每一步都带时间、输入输出、Token、成本与 Score



官方文档强调：应用 Trace 捕获 Prompt、模型响应、Token、延迟以及中间工具/检索步骤；SDK 后台批量异步发送。

Langfuse 不替代 Prometheus/DCGM：三层各自回答不同问题

层	Langfuse / LLM Trace	vLLM / Prometheus	DCGM / GPU
主要对象	Trace、Generation、Tool、Score	request histogram、queue、KV、Token	GPU/MIG、memory、power、errors
擅长回答	哪一步、哪个 Prompt/模型、花多少钱、质量如何	为什么 TTFT/TPOT 变化、引擎是否饱和	设备是否忙、满、慢、坏
高基数	request、user/session、输入输出	受控标签 + Exemplar	GPU/Pod/MIG 资源标签
典型盲区	Kernel、设备健康、精细队列状态	Prompt/工具语义与质量	请求因果、业务结果与成本
统一方式	OpenTelemetry Trace/Resource 属性 + 统一模型/部署/租户/时间语义		

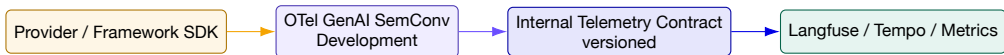
结论 产品不是层次模型。正确设计是让不同后端保留擅长的数据，再通过上下文传播实现跨层跳转。

OpenTelemetry GenAI 指标：为客户端、服务端与 Agent 提供共同语义

当前指标	含义
gen_ai.client.token.usage	客户端观察到的 input/output Token 使用量 Histogram
gen_ai.client.operation.duration	模型客户端操作时长
gen_ai.client.operation.time_to_first_chunk	客户端收到首块数据的时间
gen_ai.client.operation.time_per_output_chunk	输出块间时间
gen_ai.server.time_to_first_token	服务端 TTFT
gen_ai.server.time_per_output_token	服务端 TPOT
gen_ai.invoke_agent.inference_calls	一次 Agent 调用包含的模型调用数
gen_ai.invoke_agent.tool_calls	一次 Agent 调用包含的工具调用数
gen_ai.execute_tool.duration	工具执行时长

当前 GenAI semantic conventions 状态为 Development；字段用于建立映射思路，生产 Schema 必须版本化。

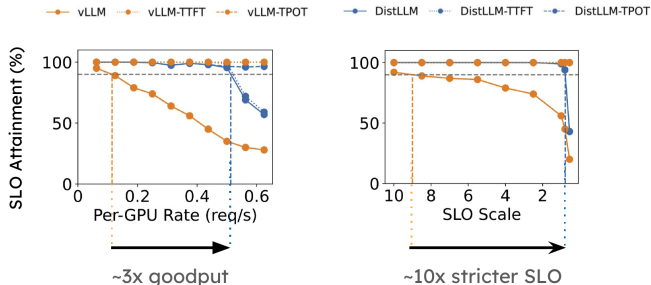
语义仍在演进：内部采集字段约定要有适配层



- ▶ 固定内部字段：task、tenant、model alias、usage、cost、outcome、release；
- ▶ 由 Collector/Processor 或 SDK adapter 映射外部语义版本；
- ▶ 保留原始 provider/model 字段，避免模型路由后身份丢失；
- ▶ Schema 变更必须有兼容测试、Dashboard 更新与迁移窗口。

Langfuse 官方说明 OTel GenAI 属性仍在演进，Langfuse 会把收到的 Span 映射到自己的数据模型，并兼容生态常见属性。

研究证据：在 SLO 约束下优化 Goodput，而不是只提高吞吐



读图要点

- ▶ 纵轴是 SLO attainment;
- ▶ 负载提高时，基线更早失守;
- ▶ 更严格 SLO 放大架构差异;
- ▶ 评价应同时检查 TTFT 与 TPOT。

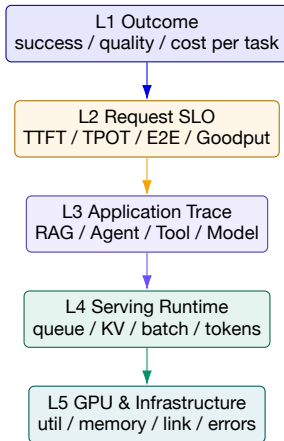
监控含义 容量面板应画 SLO attainment / Goodput, 而不只是 requests/s。

告警设计：从“单指标阈值”升级为症状 + 原因候选

告警	症状条件	诊断上下文
TTFT SLO burn Decode stall KV pressure Cost anomaly Quality regression GPU health	P95/P99 超阈且错误预算快速消耗 TPOT/ITL P99 恶化 KV 高且 waiting/TTFT 同步上升 cost/s 或 cost/task 偏离基线 segment score/feedback 下滑 XID/ECC/row remap 或降频事件	waiting、queue、prompt length、prefill、route running batch、output length、KV、DRAM、clock 长 Prompt、prefix hit、preemption、FB used model route、Token、重试、agent steps、tenant release、prompt、index、model、judge version GPU UUID、Pod、受影响 Trace、节点健康

页面策略 Page 告警优先用户影响和硬件故障；纯资源高水位可先 Ticket/自动扩容，避免告警疲劳。

推荐 Dashboard: 从结果向资源逐层下钻



首页不应堆满设备指标。值班者先判断用户/任务影响，再利用 Trace 与资源面板解释原因。

四、模型质量、RAG 与 Agent 可观测性

正确答案和成功任务不能从 CPU/GPU 指标中推导，必须建立独立但可关联的评测与反馈链。

上线前怎样评测，上线后怎样发现质量下降？



发布前 固定数据集、参考答案、规则、Judge 与基线。

发布后 Trace 抽样、用户反馈、业务 outcome 和真实失败样本。

质量标签往往延迟到达，必须保留可回填的 **Trace** 身份



- ▶ 即时代理指标：格式、引用、拒答、工具返回码；
- ▶ 延迟标签：用户点赞/改写、工单是否解决、代码是否通过测试；
- ▶ 长期标签：留存、收入、事故是否复发；
- ▶ 质量存储必须记录评测器、版本、阈值和置信度。

评测基础设施：不要只运行一次 Demo

评测集

- ▶ 100+ 样例，覆盖正常、边界、对抗和歧义；
- ▶ 按任务、语言、租户、长度、风险等级分段；
- ▶ 记录数据来源、同意与脱敏；
- ▶ 难例来自生产失败 Trace。

执行与治理

- ▶ 合并前检查 + 夜间生产抽样；
- ▶ 规则、参考答案、Schema、LLM-as-Judge；
- ▶ 结果持久化与趋势 Dashboard；
- ▶ 超过方差范围才判定回归。

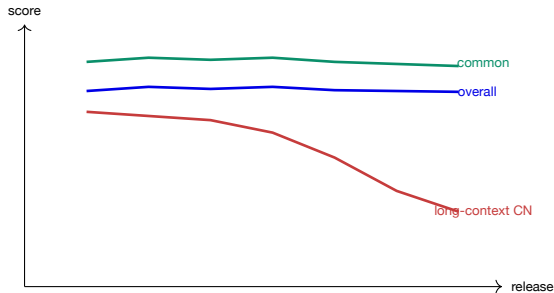
可复现性 每次结果绑定 app release、model、prompt、retrieval index、judge 和 dataset version。

LLM-as-Judge 很有用，但不是客观真值机器

风险	表现	缓解
自我偏好 位置/长度偏好 提示敏感 数据偏差 非确定性 被攻击	同模型评自己分数偏高 更长或先出现的答案得分高 评测 Prompt 小改导致分数变动 简单样例占多数 同一答案分数波动 输出中包含诱导 Judge 的文字	使用不同模型、人工校准、成对比较 随机顺序、长度归一、明确 rubric 版本化、回归测试、固定输出 Schema 分段报告、困难集、生产失败样本 重复采样、置信区间、阈值留 margin 隔离待评内容、结构化输入、对抗测试

纪律 Judge Score 是一个观测信号；重要决策需要与规则、人工抽检和业务 Outcome 交叉验证。

全局平均质量稳定，某个分段可能已经崩溃



至少切分

- ▶ 任务/意图；
- ▶ 语言与地区；
- ▶ Prompt 长度；
- ▶ 模型/路由；
- ▶ 检索索引；
- ▶ 风险等级；
- ▶ 新旧用户/租户。

单一 pass rate 会隐藏“占比小但风险高”的分段退化。

RAG 调用链：回答质量来自检索与生成的组合



检索 top-k、recall、rank、过滤、索引版本。

上下文 chunk 数、Token、重复率、上下文精度。

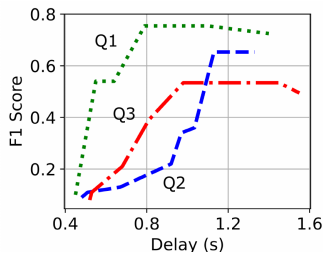
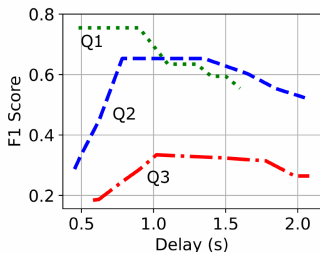
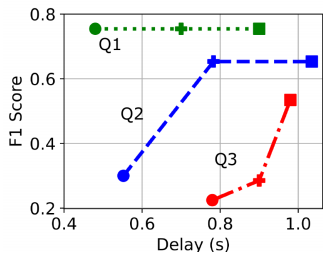
生成 TTFT、成本、groundedness、citation correctness。

RAG 质量指标分为“找得到、用得好、答得对”

层次	指标示例	需要的参考/反馈
Retrieval Context Generation	Recall@k、MRR、NDCG、命中文档率 context precision/recall、重复率、Token 数 groundedness、faithfulness、citation correctness	已知相关文档或人工标注 答案所需证据片段 检索上下文与参考事实
End-to-end Efficiency	exact match、F1、任务成功、用户反馈 retrieval latency、TTFT、总延迟、成本	参考答案或业务结果 SLO 与预算

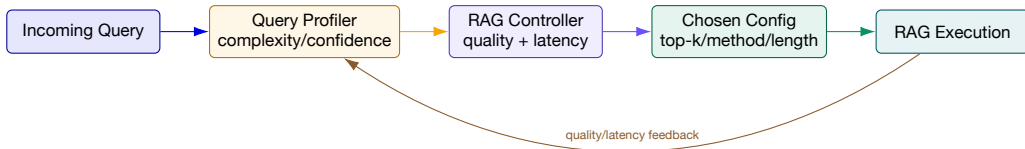
不要混淆 最终答案错，可能是没检索到、检索到但没用、或生成时编造；Trace 必须保留中间证据。

研究案例 METIS: RAG 的质量—延迟权衡随 Query 变化



读图 同一配置改变对不同 Query 的 F1 与延迟影响不同；不存在对所有请求都最优的固定 RAG 配置。

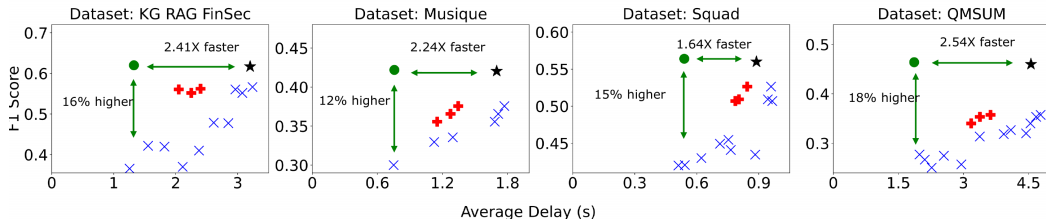
METIS: 把 Query 特征、质量预测与资源状态放进控制环



- ▶ Trace 记录 Query profile、候选配置、选中配置和控制器版本；
- ▶ 监控预测置信度，低置信度回退到安全配置；
- ▶ 质量、延迟和资源约束共同决定路由；
- ▶ 在线反馈用于更新 profiler，但需防止反馈偏差。

METIS 实验结果：同时观察质量与平均延迟

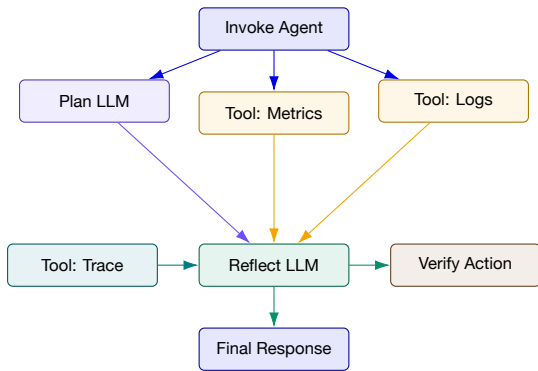
● METIS (w/ adapted RAG config and batching) + Parrot* (w/ fixed RAG config) ★ AdaptiveRAG* (selected config w/ Parrot) × vLLM (w/ fixed RAG config)



性能结论 图中不同数据集显示
1.64–2.54× 延迟改善。

质量结论 同时报告 F1 变化，避免只展示系统加速。

Agent 调用链：一次用户请求可能扩展成一棵执行树



根 Span 表示任务边界；模型、工具、检索、记忆和验证都应是可嵌套 Observation/Span。

Agent 指标：步骤数本身就是可靠性与成本信号

指标	解释	典型异常
agent duration	完整任务耗时	外部工具或循环拉长
inference calls / task	模型调用次数	规划反复、重试、反思过多
tool calls / task	工具执行次数	工具选择错误或循环
tool duration/error	外部依赖健康	API 限流、权限、超时
tokens / task	总上下文与输出消耗	历史膨胀、重复上下文
cost / successful task	单位价值成本	失败重试或昂贵模型路由
max depth / repeated edge	执行图复杂度	无限循环、状态机错误

OTel GenAI 当前提供 `invoke_agent duration`、`inference calls`、`tool calls` 与 `execute_tool duration` 等开发中指标。

如何识别 **Agent** 循环：看“结构重复”，不只看 **Token** 激增

循环证据

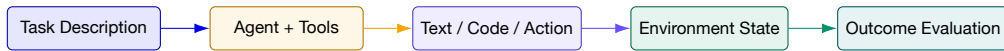
- ▶ 相同 tool name + 参数摘要重复；
- ▶ plan → tool → reflect 边重复；
- ▶ session state 没有实质变化；
- ▶ inference/tool calls 超过任务基线；
- ▶ max steps、timeout 或预算触发结束。

保护机制

- ▶ 最大步数、最大 Token、最大成本；
- ▶ 相同动作去重与幂等键；
- ▶ 环境状态变化验证；
- ▶ 高风险工具人工审批；
- ▶ 循环 Trace 自动加入失败评测集。

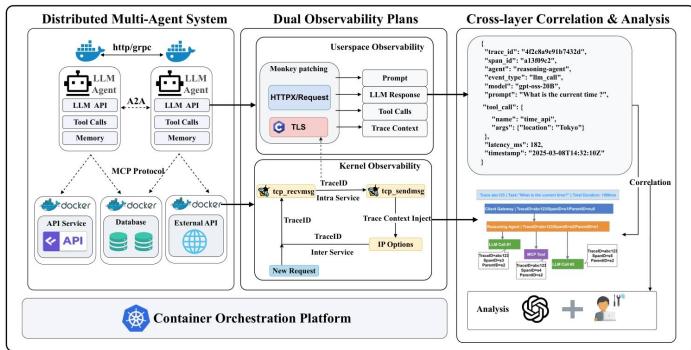
为什么 **Trace** 必需 Counter 只能说调用了 20 次；调用树能说明是哪条边反复执行、为什么没有进展。

Agent 评测要验证 Outcome，而不是只判断文字“像成功”



- ▶ 运维 Agent 说“已修复”不等于服务真的恢复；
- ▶ 需要查询 Deployment/Pod/指标/Trace 验证目标状态；
- ▶ 工具返回成功也不保证业务 SLO 恢复；
- ▶ Outcome Validity 要检查评测结果是否代表真实任务成功。

AgentTracer: 连接用户态与内核态调用链



可观测性价值

- ▶ 多 Agent / A2A / MCP;
- ▶ Prompt、响应与 Tool;
- ▶ Trace context 跨服务传播;
- ▶ 内核网络事件关联;
- ▶ 跨层分析与人工复核。

本讲连接 Agent Trace 不是只有 LLM Span，还要延伸到依赖服务和系统路径。

Prompt/Response 可观测性与隐私：默认最小化，而不是默认全量记录

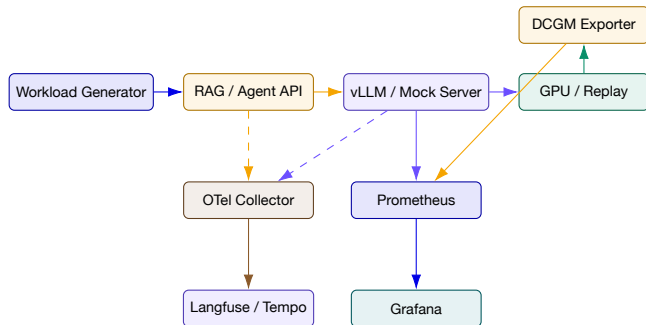
数据	风险	推荐控制
Prompt/Response	PII、机密、版权、密钥	默认不入 Metric；抽样、脱敏、加密、字段权限
Tool 参数/结果	凭证、文件内容、生产数据	allowlist 字段、摘要、secret scanner
User/Session ID	跨系统身份关联	哈希/伪匿名、租户隔离、保留期限
OTel Baggage	会跨服务甚至第三方传播	禁止密码/API key/个人数据进入 baggage
Embedding/向量	可能泄露原文语义	访问控制、数据驻留、删除策略
评测数据集	生产失败样本含敏感内容	同意、去标识化、用途限制、审计

Langfuse 使用提醒 它能记录精确 Prompt/Response，这是能力也是风险；生产启用前必须定义采样、脱敏、权限和数据驻留。

五、实验1：搭建AI推理服务可观测性体系

采集各层数据，关联到请求，注入压力，诊断原因并检查恢复。

实验架构：应用 Trace、推理指标与 GPU 指标并行采集



真实 GPU 与无 GPU 环境都可完成；Replay 数据必须明确标记为回放，不能伪装成实时硬件观测。

实验任务 0-1：先定义 SLO，再验证数据是否完整

任务 0 | 采集字段约定

- ▶ 选择 Chat、RAG 或 Agent 场景；
- ▶ 定义 TTFT、TPOT、Goodput、质量和成本阈值；
- ▶ 定义 model、tenant、release、task 等字段；
- ▶ 列出禁止记录的敏感字段。

任务 1 | 基础采集

- ▶ 抓取 vLLM /metrics；
- ▶ GPU 环境部署 dcgm-exporter；
- ▶ 应用生成 OTel Trace；
- ▶ Langfuse/Tempo 中可见嵌套调用链；
- ▶ Prometheus target 全部为 up。

实验任务 2：在同一 Trace 中记录模型、Token 与结果

```
with tracer.start_as_current_span("gen_ai.generate") as span:
    span.set_attribute("gen_ai.operation.name", "chat")
    span.set_attribute("gen_ai.request.model", model)
    span.set_attribute("app.task.type", "rag_qa")
    response = call_model(messages, stream=True)
    span.set_attribute("app.usage.input_tokens", usage.input)
    span.set_attribute("app.usage.output_tokens", usage.output)
    span.set_attribute("app.outcome", outcome)
```

Langfuse 路径 使用其 OTel-native SDK/Span Processor 或 OTLP endpoint；短任务退出前 flush/shutdown。

双后端路径 Collector fan-out 到 Langfuse 与 Tempo；统一 Trace ID，分别用于 LLM 分析和通用调用链。

示例为采集字段约定伪代码；实际属性名按所用 OTel GenAI 与 Langfuse SDK 版本适配。

实验任务 3：完成五层 Dashboard

层	必选 Panel	下钻目标
Outcome Request Application	task success、quality score、cost/success RPS、TTFT/TPOT/E2E P50/P95/P99、Goodput inference/tool calls、tool latency/error、RAG latency	失败/低分 Trace model / tenant / prompt bucket Langfuse/Tempo Trace
Runtime	running/waiting、queue/prefill/decode、KV、Token/s	vLLM instance
GPU	util、FB used/free、power/temp/clocks、XID/ECC	GPU UUID / Pod / node

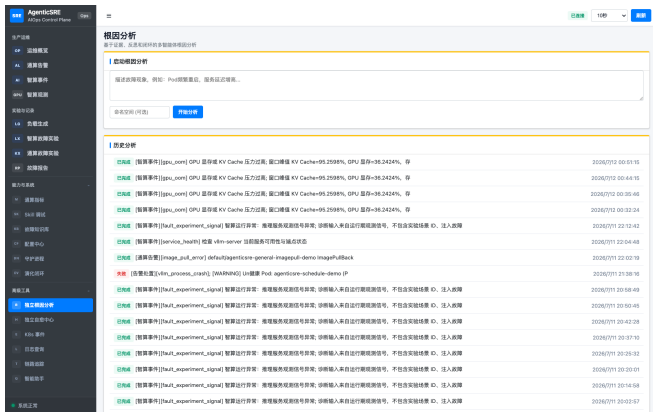
验收 从一条 P99 TTFT 曲线能跳到代表性 Trace，并最终定位到对应 vLLM Pod 与 GPU/MIG。

实验任务 4：注入四类压力，预测指标变化后再执行

注入	预期症状	关键区分证据
长 Prompt 高并发 长输出 Agent 循环 可选 GPU 故障回放	TTFT、prefill、KV 上升 waiting、queue、TTFT 上升 E2E、KV 占用、成本上升 inference/tool calls、Token/任务、成本上升 clock/XID/ECC/链路事件	input Token 分布、queue vs compute running 到达上限、Token/s 饱和 output Token、TPOT、length finish 重复调用边、状态不变、max step GPU UUID、时间窗、受影响 Trace

实验纪律 每次注入前写出假设；注入后用证据支持或反驳，不能只提交“曲线变了”的截图。

企业案例讨论: KV Cache 95% 不等于物理显存 OOM



截图中的线索

- ▶ 事件标签为 `gpu_oom`;
- ▶ KV Cache 峰值约 95.26%;
- ▶ GPU 显存约 36.24%;
- ▶ 需要区分引擎 block 压力与设备总显存。

待补充证据 列出还需查询的 waiting、TTFT、prompt length、FB used、preemption 与 Trace。

没有 GPU 也能完成实验，但必须诚实标记数据来源

CPU / Mock 路径

- ▶ OpenAI-compatible mock streaming server;
- ▶ 人为控制 queue、TTFT、TPOT 与 Token 数;
- ▶ OTel + Langfuse + Prometheus 全链路真实运行;
- ▶ GPU 指标使用课程提供的 time-series replay。

Replay 约束

- ▶ 标签增加 source="replay";
- ▶ 保留原场景与采样周期说明;
- ▶ 不用于比较本机 GPU 性能;
- ▶ 只验证查询、关联、告警与诊断逻辑。

实验目标 本实验考察可观测性工程和推理诊断，可在有限 GPU 资源下完成。

实验交付物、评分与复查要求

交付物	权重	通过条件
采集字段约定与隐私策略	15%	SLO、字段、版本、敏感数据边界明确
数据采集与调用链	25%	vLLM/DCGM 或 replay、OTel、Langfuse/Tempo 可关联
五层 Dashboard	20%	Outcome 到 GPU 逐层下钻，分位数与切片正确
故障/压力实验	25%	至少 3 类注入，有预测、证据、反证和恢复验证
诊断报告	15%	引用 Trace/查询/对象/时间窗，说明限制与替代假设

禁止只提交整页 Dashboard 截图；报告必须写清“看到什么一推断什么一如何验证”。

成本归因算例：一次成功任务实际花了多少钱？

假设每千输入 Token 计 1 成本单位、每千输出计 4 单位；工具每次计 0.2 单位，无缓存折扣。

步骤	输入 / 输出 Token	模型成本	工具成本
规划	1,000 / 200	$1.0 + 0.8 = 1.8$	0
工具失败后重试	500 / 100	$0.5 + 0.4 = 0.9$	0.4（两次调用）
答案生成	2,000 / 300	$2.0 + 1.2 = 3.2$	0
任务合计	3,500 / 600	5.9	0.4

$$C_{task} = 5.9 + 0.4 = 6.3 \text{ 成本单位}$$

推导与判断 只看最后一次模型调用会把成本记成 3.2。任务 ID 应关联全部模型调用、失败尝试和工具费用；这里使用虚构计价，不对应任何厂商报价。

算例使用假设数据。

参考资料与延伸阅读

- ▶ vLLM Documentation, *Metrics* (stable): 服务级与请求级生产指标;
- ▶ NVIDIA DCGM Exporter Documentation 与官方 `default-counters.csv`;
- ▶ OpenTelemetry Semantic Conventions for Generative AI Metrics (Development);
- ▶ Langfuse Documentation: Observability Overview、Core Concepts、Token & Cost、OTel Integration;
- ▶ Zhong et al., *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving*, OSDI 2024;
- ▶ Kwon et al., *Efficient Memory Management for Large Language Model Serving with PagedAttention*, SOSP 2023;
- ▶ Xu et al., *TELLER*, ASE 2026; AgentTracer / AgenticSRE 系统原型;
- ▶ METIS 参考材料: query-level RAG quality-latency adaptation;
- ▶ 本地 `ai-infra-engineer-learning-main`: LLM observability、cost monitoring、evaluation harness。

官方链接: [vLLM Metrics](#); [DCGM Exporter](#); [OTel GenAI SemConv](#); [Langfuse Observability](#)

让每个Token、每次
工具调用和每块GPU
都能回到一个可
验证的用户任务

校企联合研究生课程 面向 AI 原生系统的智能运维